

Laporan Praktikum Kontrol Cerdas

Minggu Ke-3

Nama : Rani Dwi Setiani
NIM : 224308043
Kelas : TKA-6B
Akun Github : <https://github.com/Rani-TKA6B>
Student Lab Assistant : Mas Alfian

1. Judul Percobaan: *Deep Learning for Intelligent ControlSystem*
2. Tujuan Percobaan:
 - Memahami konsep dasar *Deep Learning* dalam sistem kendali
 - Mengimplementasikan *Convolutional Neural Network* (CNN) untuk klasifikasi objek
 - Menggunakan *TensorFlow* dan Keras untuk membangun model *Deep Learning*.
 - Mengintegrasikan model CNN dengan *Computer Vision* untuk deteksi objek secara *real-time*
 - Menggunakan dataset dari Kaggle untuk pelatihan model
 - Mengembangkan mode *Night Vision* untuk deteksi objek dalam kondisi pencahayaan rendah.

3. Landasan Teori:

Deep Learning adalah subbidang mesin pembelajaran yang meniru cara kerja otak manusia dalam memproses data dan menciptakan pola untuk digunakan dalam pengambilan keputusan. Salah satu arsitektur utama dalam *deep learning* adalah *Convolutional Neural Network* (CNN), yang dirancang khusus untuk mengolah data berbentuk grid, seperti citra. CNN telah terbukti efektif dalam tugas-tugas seperti klasifikasi objek, deteksi objek, dan segmentasi citra. CNN bekerja dengan menerapkan operasi konvolusi pada citra input untuk mengekstraksi fitur-fitur penting, seperti tepi, tekstur, dan pola kompleks lainnya. Lapisan-lapisan konvolusi ini memungkinkan model untuk mempelajari representasi hierarkis dari data visual, yang sangat berguna dalam

tugas klasifikasi objek. Dalam penelitian Wahyudi Setiawan, dijelaskan bahwa CNN memiliki kemampuan yang signifikan dalam pengenalan citra dan telah diterapkan dalam berbagai bidang, termasuk medis, pertanian, industri, dan robotika (Setiawan, 2021). Penggunaan TensorFlow dan Keras dalam Membangun Model *Deep Learning*, TensorFlow dan Keras adalah dua alat utama yang digunakan dalam pengembangan model deep learning. TensorFlow, dikembangkan oleh Google Brain, adalah pustaka *open-source* untuk komputasi numerik dan pembelajaran mesin. Keras, yang sekarang menjadi bagian dari TensorFlow, menyediakan antarmuka tingkat tinggi yang memudahkan perancangan dan pelatihan model jaringan saraf. Penggunaan Keras dengan TensorFlow memungkinkan pengembang dengan mudah membangun dan melatih model CNN untuk berbagai aplikasi, termasuk klasifikasi citra. Integrasi CNN dengan *Computer Vision* untuk mendeteksi objek secara *real-time*, Integrasi CNN dengan teknik *computer vision* memungkinkan deteksi objek secara *real-time*. Algoritma seperti *You Only Look Once* (YOLO) dan *Faster R-CNN* telah dikembangkan untuk mendeteksi dan mengklasifikasikan objek dalam citra atau video dengan kecepatan tinggi dan akurasi yang tinggi. Dalam penelitian yang berjudul “Perancangan Sistem Deteksi Objek Berbasis *Convolutional Neural Network* Menggunakan YOLOv4 dan OpenCV”, dijelaskan bahwa sistem deteksi objek ini didasarkan pada CNN dan mampu mendeteksi objek secara *real-time*, yang dapat diterapkan pada sistem robotik semi-otonom (Baskoro dkk., 2022). Mode *Night Vision* untuk mendeteksi objek dalam kondisi pencahayaan rendah, deteksi objek dalam kondisi pencahayaan rendah atau malam hari merupakan tantangan tersendiri. Penggunaan mode *night vision* atau teknik pengolahan citra inframerah dapat membantu sistem dalam mendeteksi objek di lingkungan dengan cahaya minimal. Dengan menggabungkan teknologi ini dengan CNN, sistem dapat mengenali objek bahkan dalam kondisi pencahayaan yang buruk. Meskipun penelitian spesifik mengenai implementasi *night vision* dengan CNN belum banyak dipublikasikan, pendekatan ini memiliki potensi besar dalam aplikasi keamanan dan pengawasan.

Machine Learning adalah salah satu bidang cabang ilmu komputer yang memberikan kemampuan kepada komputer untuk dapat belajar tanpa diprogram secara eksplisit (Setiawan, 2021). Untuk bisa mengaplikasikan teknik-teknik machine learning maka harus ada data. Tanpa data maka algoritma machine learning tidak dapat bekerja. Data yang ada biasanya dibagi menjadi dua, yaitu data training dan data testing. Data training digunakan untuk melatih algoritma, sedangkan data *testing* digunakan untuk mengetahui performa algoritma yang telah dilatih sebelumnya ketika menemukan data baru yang belum pernah dilihat. *Machine Learning* sendiri terbagi menjadi beberapa algoritma yang berbeda-beda dan setiap dari algoritma tersebut memiliki fungsi dan tujuannya masing-masing, seperti *supervised learning*, *unsupervised learning*, dan *reinforcement learning* (Karimah, 2023). *Supervised Learning* adalah algoritma yang menyimpulkan fungsi dari data pelatihan berlabel yang terdiri dari sekumpulan contoh pelatihan, misalnya seperti *logistic regression* dan *K-Nearest Neighbors* (KNN). *Unsupervised Learning* adalah algoritma yang bertujuan untuk meningkatkan kinerja tugas pembelajaran yang diawasi dimana kita tidak memiliki banyak data, contohnya seperti *k-means*, *Apriori*, DBSCAN, dan *clustering*. *Reinforcement Learning* adalah algoritma yang mengumpulkan pengetahuan (sinyal penguatan) untuk memilih tindakan yang mengarah ke hasil tertinggi yang diharapkan. Dibandingkan dengan metodologi lain dalam *Machine Learning*, algoritma *Reinforcement Learning* memiliki kelebihan yang berbeda yaitu kemampuan untuk secara mandiri menjelajahi lingkungan yang sangat dinamis dan stokastik dan mengembangkan, dengan mengumpulkan umpan balik evaluatif dari lingkungan, kebijakan kontrol yang optimal (Fathurohman, 2021). *Machine Learning* memungkinkan sistem kendali untuk belajar dari data dan meningkatkan performanya seiring waktu. Beberapa contoh penerapannya yaitu *Predictive Maintenance* (mendeteksi potensi kegagalan sistem sebelum terjadi), *Adaptive Control* (sistem kendali yang dapat menyesuaikan parameter secara otomatis), dan *Anomaly Detection* (mendeteksi perilaku tidak normal dalam sistem industri) (Roihan dkk., 2020). Penggunaan CNN dalam sistem kontrol cerdas menawarkan kemampuan yang kuat dalam klasifikasi dan

deteksi objek. Dengan dukungan alat seperti TensorFlow dan Keras, pengembangan model *deep learning* menjadi lebih terjangkau dan efisien. Integrasi dengan teknik *computer vision* memungkinkan deteksi objek secara *real-time*, sementara penerapan mode night vision membuka peluang untuk operasi dalam kondisi pencahayaan rendah. Pengembangan lebih lanjut dalam area ini diharapkan dapat meningkatkan kinerja dan aplikasi sistem kontrol cerdas di berbagai bidang.

4. Analisis dan Diskusi

- Analisis

Hasil percobaan menunjukkan bahwa model CNN yang dikembangkan mampu mengenali kondisi pencahayaan dengan tingkat akurasi yang cukup tinggi. Implementasi *real-time* menggunakan *TensorFlow* memungkinkan sistem untuk mendeteksi dan menyesuaikan pengolahan gambar dengan cepat terhadap perubahan pencahayaan di lingkungan sekitar. Salah satu tantangan utama dalam penelitian ini adalah menangani kondisi pencahayaan yang sangat rendah atau terlalu terang, yang dapat menyebabkan kehilangan detail pada gambar. Oleh karena itu, diperlukan teknik tambahan seperti penggunaan *luxmeter* untuk meningkatkan akurasi deteksi. Selain itu, performa model sangat bergantung pada kualitas dan keragaman dataset yang digunakan dalam pelatihan. Jika dataset kurang representatif, model dapat mengalami kesulitan dalam mengenali kondisi pencahayaan yang tidak terdapat dalam data pelatihan. Oleh karena itu, peningkatan dataset dengan variasi pencahayaan yang lebih luas dapat meningkatkan kemampuan generalisasi model.

- Diskusi

Keunggulan utama dari sistem ini adalah kemampuannya untuk beroperasi secara *real-time*, yang membuka peluang aplikasi di berbagai bidang seperti kendaraan otonom dan sistem pengawasan keamanan. Namun, untuk aplikasi di dunia nyata, masih perlu dilakukan optimasi dalam hal efisiensi komputasi, terutama jika sistem diterapkan pada perangkat dengan keterbatasan daya dan sumber daya pemrosesan. Selain itu, pendekatan ini dapat dikombinasikan dengan teknik *machine learning* lainnya, seperti *reinforcement learning*, untuk meningkatkan adaptasi sistem terhadap lingkungan yang dinamis. Dari sudut

pandang industri, implementasi sistem night vision berbasis deep learning dapat memberikan nilai tambah dalam berbagai sektor, termasuk transportasi, keamanan, dan teknologi militer. Namun, untuk mencapai tingkat akurasi yang lebih tinggi, diperlukan penelitian lebih lanjut dalam hal optimasi arsitektur model dan penggunaan hardware yang lebih efisien.

5. *Assigment:*

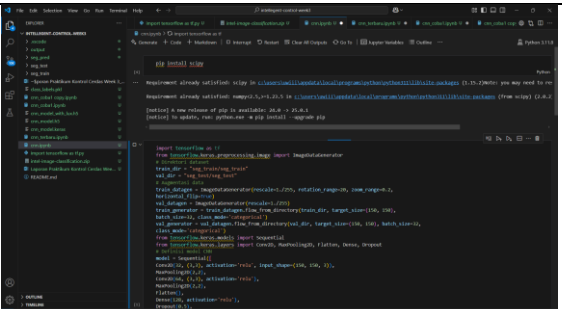
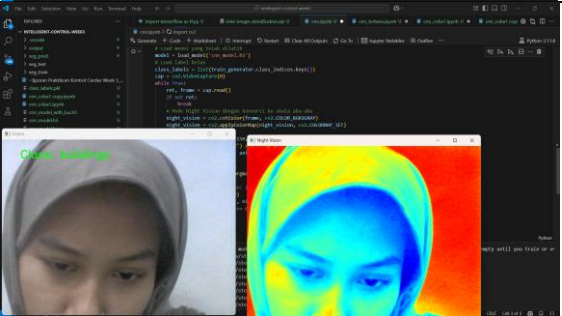
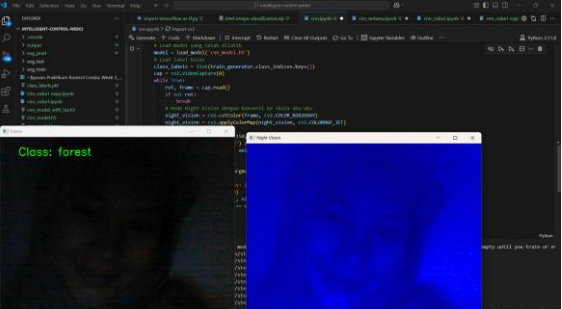
Dalam era teknologi modern, sistem kontrol cerdas (*Intelligent Control System*) semakin banyak digunakan untuk meningkatkan efisiensi dan keakuratan dalam berbagai bidang, termasuk kendaraan otonom, keamanan, dan pengawasan lingkungan. Salah satu pendekatan yang banyak diterapkan adalah pemanfaatan deep learning, khususnya *Convolutional Neural Network* (CNN) dalam pengolahan citra. CNN telah terbukti sangat efektif dalam mendeteksi dan mengenali pola visual, sehingga banyak digunakan dalam aplikasi *Computer Vision*. Dalam penelitian ini, dilakukan implementasi CNN menggunakan *TensorFlow* secara *real-time* untuk mengembangkan mode night vision yang dapat mendeteksi kondisi pencahayaan sekitar. Sistem ini bertujuan untuk meningkatkan kemampuan komputer dalam mengidentifikasi dan menyesuaikan dirinya terhadap berbagai tingkat pencahayaan, baik dalam kondisi terang maupun gelap. Teknologi ini dapat diterapkan dalam berbagai bidang, seperti navigasi kendaraan otonom di malam hari, sistem pengawasan berbasis visi komputer, serta aplikasi keamanan yang membutuhkan adaptasi terhadap lingkungan dengan pencahayaan yang bervariasi, mengumpulkan dataset gambar dari berbagai kondisi pencahayaan, termasuk siang, senja, dan malam, menggunakan kamera *real-time* untuk menangkap perubahan pencahayaan lingkungan, mengonversi gambar ke dalam format *grayscale* atau menggunakan filter khusus untuk meningkatkan kontras dalam kondisi pencahayaan rendah, ormalisasi data agar model dapat mengenali pola dengan lebih baik, menggunakan arsitektur CNN dengan beberapa lapisan konvolusi untuk mendeteksi fitur pencahayaan dalam gambar, melatih model menggunakan *TensorFlow* dengan dataset yang telah dikumpulkan, menghubungkan model dengan kamera *real-time* untuk mendeteksi kondisi pencahayaan langsung, menggunakan *TensorFlow* untuk menjalankan

inferensi secara efisien di perangkat keras yang tersedia, mengukur akurasi model dalam mendeteksi kondisi pencahayaan, melakukan uji coba dalam berbagai lingkungan untuk memastikan keandalan sistem. Dengan adanya implementasi ini, diharapkan sistem kontrol cerdas dapat beradaptasi lebih baik terhadap lingkungan dengan pencahayaan yang dinamis, sehingga meningkatkan efisiensi dan keamanan dalam berbagai aplikasi yang bergantung pada *Computer Vision*.

- Sebelum Modifikasi

Program ini adalah implementasi *Convolutional Neural Network* (CNN) menggunakan *TensorFlow* dan Keras untuk melakukan klasifikasi gambar berdasarkan dataset yang disimpan dalam direktori tertentu

6. Data dan Output Hasil Pengamatan

No.	Variabel	Hasil Pengamatan
Sebelum Modifikasi		
	Coding	
1.	Dalam ruangan (keadaan lampu ON)	
2	Dalam ruangan (keadaan lampu OFF)	

Sesudah Modifikasi Coding

No	Kondisi	Gambar																
1	Dalam ruangan (keadaan lampu ON)																	
	Grafik	<table border="1" style="margin-top: 10px;"> <caption>Data for Grafik Intensitas Cahaya (Lux)</caption> <thead> <tr> <th>Waktu (HH-MM-SS)</th> <th>Nilai LUX</th> </tr> </thead> <tbody> <tr><td>21:30:55</td><td>173</td></tr> <tr><td>21:30:56</td><td>203</td></tr> <tr><td>21:30:57</td><td>239</td></tr> <tr><td>21:30:58</td><td>238</td></tr> <tr><td>21:30:59</td><td>234</td></tr> <tr><td>21:31:00</td><td>233</td></tr> <tr><td>21:31:01</td><td>233</td></tr> </tbody> </table>	Waktu (HH-MM-SS)	Nilai LUX	21:30:55	173	21:30:56	203	21:30:57	239	21:30:58	238	21:30:59	234	21:31:00	233	21:31:01	233
Waktu (HH-MM-SS)	Nilai LUX																	
21:30:55	173																	
21:30:56	203																	
21:30:57	239																	
21:30:58	238																	
21:30:59	234																	
21:31:00	233																	
21:31:01	233																	
2.	Dalam ruangan (keadaan lampu OFF)																	

```

1  #include <iostream>
2  #include <random>
3  using namespace std;
4  int main()
5  {
6      int randNum;
7      randNum = rand() % 100;
8      cout << randNum << endl;
9      return 0;
10 }
```

The screenshot shows a Windows desktop with three open applications:

- Terminal (cmd.exe):** Displays a list of system metrics for a Raspberry Pi 4 Model B. The metrics include CPU usage (100%), memory usage (100%), and various network and disk I/O statistics. The output is as follows:


```

      C:\Users\user>systeminfo
      <pre>
      </pre>
      </div>
      <div data-bbox="31 450 330 610" data-label="Image">
        <img alt="Screenshot of a web browser showing a car image." data-bbox="31 450 330 610"/>
        A screenshot of a web browser window displaying a car image. The text "Class: mountain" is visible at the top, followed by "Price: 120,000 (Sedan)" and "Time: 0:30:58".
      </div>
      <div data-bbox="31 610 330 780" data-label="Figure">
        <img alt="Screenshot of a heatmap visualization." data-bbox="31 610 330 780"/>
        A screenshot of a heatmap visualization showing a circular pattern of colors ranging from blue to red, indicating a distribution of values. The title of the window is "Heatmap".
      </div>
```

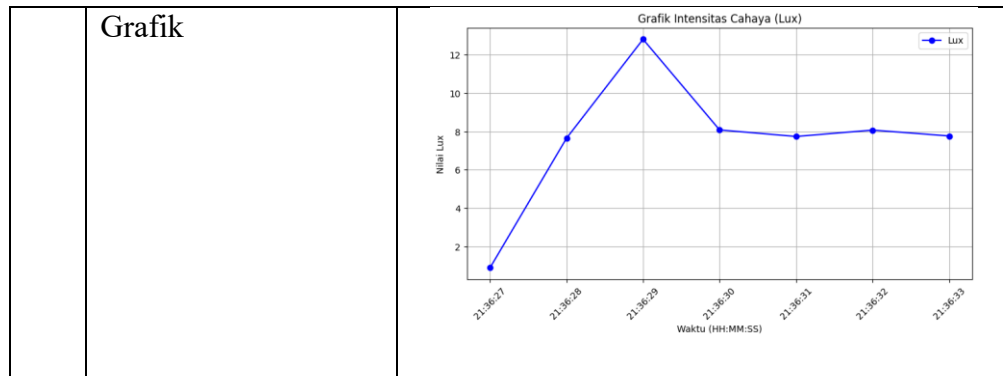
Waktu (HH:MM:SS)	Nilai Lux
21:30:55	174
21:30:56	203
21:30:57	240
21:30:58	239
21:30:59	235
21:31:00	235
21:31:01	235

The screenshot displays a JupyterLab environment. On the left, a sidebar shows the file explorer with folders like 'environment.yml' and 'jupyterlab'. The main area is split into two panes. The top pane is a terminal window showing the output of a Keras model's predict function. The output indicates a predicted class of 'forest' with a Lux score of 10.60 and a Gelap score of 21:36:29. The bottom pane is a code editor showing a Keras model definition. The model is a Sequential model with three layers: a Dense layer with 100 units, a Dense layer with 100 units, and a Dense layer with 1 unit. The model is compiled with the Adam optimizer and categorical_crossentropy loss function. The model is then trained on a dataset of 1000 samples, with a validation set of 100 samples. The training process shows a decrease in loss and an increase in accuracy over 10 epochs.

```

class: forest
Lux: 10.60 (Gelap)
Time: 21:36:29

Model: "Sequential"
Layer (type)                 Output Shape         Connected to          Training
                                                                     Mode
=====================================================================
Dense (Dense)                (100,)               Dense1[0]              train
Dense1 (Dense)                (100,)               Dense2[0]              train
Dense2 (Dense)                (1,)                 Dense3[0]              train
Activation (Activation)       (1,)                 Dense3[0]              train
Loss (Loss)                   (1,)                 Dense3[0]              train
Optimizer (Optimizer)         (1,)                 Dense3[0]              train
Total params: 10,100
Trainable params: 10,100
Non-trainable params: 0
Total size: 40.4 KB
Trainable size: 40.4 KB
1000 samples, 100 validation samples
Epoch 1/10
1000/1000 [100%] 1.0000e+00 1.0000e+00 0.0000e+00 0.0000e+00
Epoch 2/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 3/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 4/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 5/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 6/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 7/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 8/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 9/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 10/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
1000 samples, 100 validation samples
Epoch 1/10
1000/1000 [100%] 1.0000e+00 1.0000e+00 0.0000e+00 0.0000e+00
Epoch 2/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 3/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 4/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 5/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 6/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 7/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 8/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 9/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 10/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
1000 samples, 100 validation samples
Epoch 1/10
1000/1000 [100%] 1.0000e+00 1.0000e+00 0.0000e+00 0.0000e+00
Epoch 2/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 3/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 4/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 5/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 6/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 7/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 8/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 9/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 10/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
1000 samples, 100 validation samples
Epoch 1/10
1000/1000 [100%] 1.0000e+00 1.0000e+00 0.0000e+00 0.0000e+00
Epoch 2/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 3/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 4/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 5/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 6/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 7/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 8/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 9/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 10/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
1000 samples, 100 validation samples
Epoch 1/10
1000/1000 [100%] 1.0000e+00 1.0000e+00 0.0000e+00 0.0000e+00
Epoch 2/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 3/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 4/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 5/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 6/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 7/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 8/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 9/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 10/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
1000 samples, 100 validation samples
Epoch 1/10
1000/1000 [100%] 1.0000e+00 1.0000e+00 0.0000e+00 0.0000e+00
Epoch 2/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.0000e+00
Epoch 3/10
1000/1000 [100%] 0.9999e+00 0.9999e+00 0.0000e+00 0.
```



7. Kesimpulan:

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

- implementasi CNN menggunakan TensorFlow dalam sistem kontrol cerdas berbasis computer vision terbukti efektif dalam mendeteksi dan menyesuaikan diri terhadap berbagai kondisi pencahayaan secara real-time
- Mode night vision yang dikembangkan mampu meningkatkan visibilitas dalam lingkungan dengan pencahayaan rendah, sehingga dapat diterapkan dalam berbagai aplikasi
- Evaluasi terhadap model menunjukkan bahwa akurasi deteksi pencahayaan dapat ditingkatkan lebih lanjut dengan optimasi arsitektur dan dataset yang lebih besar.

8. Saran:

- Menggunakan dataset yang lebih luas dan beragam untuk meningkatkan generalisasi model.
- Mengimplementasikan teknik peningkatan citra (image enhancement) yang lebih canggih, seperti histogram equalization atau deep learning-based denoising.
- Mengintegrasikan model dengan sistem embedded atau edge computing agar lebih efisien dalam implementasi di perangkat dengan keterbatasan daya dan sumber daya komputasi.
- Menguji sistem dalam skenario dunia nyata dengan berbagai kondisi lingkungan untuk meningkatkan ketahanan dan akurasi model.

Dengan pengembangan lebih lanjut, sistem ini dapat menjadi solusi yang lebih andal dan adaptif dalam berbagai kondisi pencahayaan.

Daftar Pustaka:

Setiawan, W. (2021). *Pembelajaran Mendalam menggunakan Convolutional Neural Network: Teori dan Aplikasi* . Media Nusa Kreatif (MNC Publishing).

Adhisatria, MIB, & Wahyudi. (2025). Perancangan Kontrol Adaptif Kepadatan Arus Lalu Lintas dengan Metode Convolutional Neural Network. *SIKLOTRON*.

Implementasi Deep Learning dengan Metode Convolutional Neural Network (CNN) pada Pengenalan Objek. (2020). *Semantik*.

Perancangan Sistem Deteksi Objek Berbasis Convolutional Neural Network Menggunakan YOLOv4 dan OpenCV. (2023). *Sementara*.

Klasifikasi Citra Menggunakan Convolutional Neural Network (CNN) pada Caltech 101. (2019). *Jurnal Ilmu Komputer dan Informasi*.